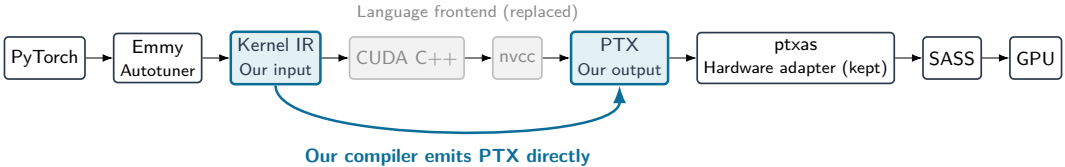
Emmy Hackathon 2026

Kernel IR $\rightarrow$ PTX, directly

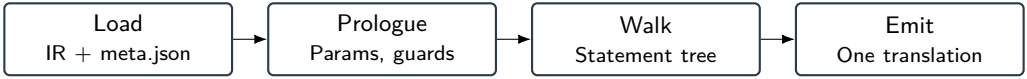
Ratan Kokal

Skip the CUDA C++ step



- The autotuner compiles thousands of variants per kernel, so the middle step's cost is paid thousands of times.
- The IR arrives fully scheduled. A strict contract means even a lookup table can emit valid PTX.

What we built



One kernel in, one PTX module out.

Kernel IR	Emitted PTX

`arg_order from meta.json`	`.visible .entry k_sum_1_reduce(`
	`  .param .u64 param_x,`
	`  .param .u64 param_sum_1)`

`in0 = load x[a0]`	`ld.global.f32 %f2, [%rd6];`
`f32 acc0 <- add(acc0, in0)`	`add.rn.f32 %f1, %f1, %f2;`
`WarpShuffle(acc0, length=32)`	`shfl.sync.bfly.b32 %r8, %r7, 16, 0x1f, %r6;`

- Fresh registers for every value, so a stale read cannot happen
- Narrow types computed in f32, same as Emmy's own pipeline
- Unsupported statements raise loudly instead of guessing

Verification engine

Verifier	Approach	Goal	Can	Cannot	Caught
Official runner	Sampled	The exact judging pipeline	Anything breaking a real kernel	Ops the 15 problems never test	Inverted tanh, error 2.0
ptxas sweep	Exhaustive	Every op × dtype assembles	Invalid PTX, wrong types	Valid PTX, wrong math	mul.s32 is not an instruction
Semantic sweep	Sampled	Every op × dtype vs numpy	Quietly wrong numbers	Ops interacting in one kernel	pow(-2,3) gave NaN, not -8
Differential vs Emmy	Sampled	Two compilers, same inputs	Divergence in output numbers	Shared misreadings	m02 reused one cached slab
Verification engine	Proof-based	Proof conditions per compile	Broken SSA, stale caches	Wrong math in valid discipline	Nothing, 100 problems clean
Thread map proof	Proof-based	Each thread maps to its cell	Wrong thread assignments	Runtime-only failures	a02 hit wrong cells before remap